Project: Distance Vector Routing with ESP32 and MicroPython

Prof. Dimitrios Zorbas

I. Introduction

Lab 5 (Part I) introduced periodic advertisements and hop-count propagation, while Lab 5 (Part II) used the selected parent to forward sensor measurements to the root. Lab 6 introduced AES encryption and data integrity. In this assignment, you will develop a complete, threaded implementation of a simple root-oriented Distance Vector (DV) routing protocol over ESP-NOW and compare it experimentally with flooding.

Every non-root device must discover its neighbors, select a parent using received signal strength (RSS, measured through RSSI in dBm) and hop count, and deliver its own and other devices' sensor data to the root. All routing information must be learned from received advertisements. Do not hard-code parents or routes. This project implements routing toward one root; routes between arbitrary pairs of devices are outside its scope.

II. Equipment and configuration

Use ESP32 boards running MicroPython with ESP-NOW and `_thread` support, and the Thonny IDE. Use at least eleven boards: one root and ten non-root devices. Each team will be responsible for one device. Give the root ID 0 and each other device a unique ID from 1 to 254; reserve 255 for broadcast. Configure every board on the same Wi-Fi channel. **All communications are broadcast-based:** send every advertisement and every data packet, including DV forwarding, as an ESP-NOW broadcast. Device IDs in packet headers provide application-level addressing. Do not maintain neighbor MAC-address mappings or register individual neighbors as unicast peers; configure only the common broadcast destination required by the ESP-NOW API.

Each non-root device must acquire data from at least one configured sensor: DHT11 (temperature and relative humidity), DS18-series temperature sensor (e.g., DS18B20), GPS (longitude and latitude), smoke detector, or accelerometer. Across the network, demonstrate all five sensor types. Document the sensor model, wiring, acquisition settings, and units for each device. For a DS18B20 wired as in Part II, use VDD to 3V3, GND to GND, and DQ to GPIO 4, and respect its configured conversion delay (up to 750 ms at 12 bits).

Provide configuration parameters for device ID, root/non-root role, routing mode (DV or flooding), enabled sensor types, advertisement interval, data interval, RSS threshold, and initial time-to-live (TTL). Suggested starting values are 5 s, 10 s, -85 dBm, and TTL 10, respectively. These are experimental settings, not universal optimum values. Apply small random timing jitter to reduce simultaneous transmissions.

III. Task 1: Neighbor discovery

Develop a periodic advertisement mechanism. Unlike Part I, every device must advertise even when it has no route, so that neighbors can discover it. Use hop value 255 to represent an unknown/unreachable root. The root always advertises hop 0 and never selects a parent.

Maintain a neighbor table containing at least:

- neighbor ID;
- advertised hop count and parent ID;
- most recent RSSI and a smoothed RSSI estimate;
- last time an advertisement was received.

Update the table whenever a valid advertisement arrives. Obtain RSSI from the received sender's ESP-NOW metadata, according to the firmware API. Associate the RSSI of each received advertisement with its advertised source ID. Any sender metadata needed by the firmware to look up RSSI is used only for that lookup; it is not part of the routing state or an addressing requirement.

Choose and document an RSS smoothing method, such as an average over recent advertisements or a moving average. Expire a neighbor when no advertisement has arrived within the configured timeout. Use `time.ticks_diff()` for elapsed-time comparisons. A stale neighbor must not remain a parent candidate.

Demonstration: Show discovery of a newly powered device, updates to its RSS estimate, and removal of a device that is switched off. Print the neighbor table periodically.

IV. Task 2: Parent selection

For a neighbor j advertising hop count h_j, the candidate distance to the root is h_j+1. Implement the following selection policy:

1) Exclude stale neighbors, neighbors with unknown routes, yourself, and neighbors advertising you as their parent. Exclude links whose smoothed RSS is below the configured threshold. Reject candidate distances that cannot be represented as a finite hop value.
2) Among the remaining candidates, prefer the smallest candidate hop count.
3) Among candidates with the same hop count, prefer the strongest smoothed RSS. Remember that -60 dBm is

stronger than -80 dBm. Break an exact tie using the smaller device ID.

4) To reduce oscillation, keep an eligible current parent when a same-hop alternative is only slightly stronger. Use a configurable RSS margin (for example, 5 dB). A shorter eligible route takes priority; an ineligible parent must be replaced or invalidated immediately.

Store the selected parent ID and your own hop count as one consistent routing state. Your advertised hop count is your parent's advertised hop count plus one. Recompute the route when advertisements arrive or neighbors expire. Support routes becoming *worse* as well as better; do not retain the "update only for a smaller hop count" rule of Part I. If no candidate exists, clear the parent and advertise hop 255.

Include the parent ID in advertisements to reduce immediate two-node loops. This does not guarantee that longer loops cannot occur during topology changes. Use bounded hop values and data TTL to contain their effects, and discuss this limitation in your report.

Demonstration: Establish a route with at least two hops. Then demonstrate a choice between two same-hop parents based on RSS, and a parent change or route invalidation after a failure. Log the old/new parent, hop count, RSS, and reason for each change.

V. Task 3: Data generation and delivery

Each non-root device must periodically generate sensor-data packets and forward them toward the root. A packet carries one sensor type; a device with multiple enabled sensors generates a separate packet for each type. Advertisements and forwarding must continue during sensor acquisition, including the two-second accelerometer sampling window.

Use the following binary inner-packet formats. These packets must be protected by the mandatory encryption envelope described below before transmission; the lengths listed here exclude security overhead. Numbers in parentheses are field lengths in bytes. Use network/big-endian byte order on every device (for example, with `struct`). Floating-point fields use four-byte IEEE 754 binary32 values.

Advertisement, type 0:

```
ptype(1) | dest_id(1) | src_id(1)
| hop(1) | parent_id(1)
```

Data, type 2:

```
ptype(1) | dest_id(1) | origin_id(1)
| seq(2) | ttl(1) | sensor_code(1)
| payload(variable)
```

Advertisements use destination 255; their source is the immediate advertiser. Parent ID 255 means no parent (including at the root). In DV mode, the data destination is the *next-hop* ID used for receiver-side filtering of a broadcast; in flooding mode it is 255. The field `origin_id` identifies the device that generated the measurement and must remain unchanged during forwarding. The one-byte `sensor_code` is part of the data packet header and uniquely identifies the sensor type and its payload format. It is distinct from `ptype`, which distinguishes advertisements from data. The data header is seven bytes; use the following fixed code assignments and payload field order:

- **Code 1 — DHT11:** temperature in degrees Celsius (float32), followed by relative humidity in percent (float32). Payload: 8 bytes; inner packet: 15 bytes.
- **Code 2 — DS18:** temperature in degrees Celsius (float32). Payload: 4 bytes; inner packet: 11 bytes.
- **Code 3 — GPS:** longitude, followed by latitude, both in decimal degrees (two float32 values). Use negative values for west/south. Payload: 8 bytes; inner packet: 15 bytes.
- **Code 4 — Smoke detection:** one unsigned byte: 0 means no smoke detected, 1 means smoke detected. Normalize the detector output to this convention and document the detection threshold or digital-output interpretation. Payload: 1 byte; inner packet: 8 bytes.
- **Code 5 — Accelerometer:** two-second average acceleration magnitude in $\mathrm{m/s^2}$ (float32). Payload: 4 bytes; inner packet: 11 bytes.

For the accelerometer, sample all three axes at a documented, approximately uniform rate over a complete two-second window. Convert each axis to $\mathrm{m/s^2}$ and compute

$$\overline{a} = \frac{1}{N}\sum_{k=1}^{N}\sqrt{a_{x,k}^2 + a_{y,k}^2 + a_{z,k}^2}.$$

Transmit this average of the per-sample magnitudes, including gravity; do not take the magnitude of the averaged axes. Report the sampling rate and actual sample count N. Use a completed window for each generated accelerometer packet.

Codes 0 and 6–255 are reserved and must be rejected. Validate the exact packet length for the sensor code, finite floating-point values, humidity in $[0, 100]$, longitude in $[-180, 180]$, latitude in $[-90, 90]$, smoke state in $\{0, 1\}$, and nonnegative acceleration magnitude. Do not encode failed sensor reads or an invalid GPS fix as valid zero measurements: log and count acquisition failures separately, and generate no data packet for a failed acquisition.

Increment one device-wide sequence number for every newly generated data packet, across all sensor types. A DHT11 or GPS payload containing two values counts as one sample/packet. The pair `(origin_id, seq)` identifies a data packet. Keep experiments short enough to avoid sequence wraparound, and reset logs and duplicate caches together between runs; do not reboot sources during a measured run.

Implement the following behavior:

- Broadcast locally generated data with `dest_id` set to the selected parent's ID.
- Process a DV data packet only when its destination is your ID. All other receivers, including the root, must ignore overheard DV data addressed to another device, without caching or delivering it. Validate packet type, length, and fields before processing.

- At an intermediate device, retain the origin, sequence, sensor code, and payload bytes, set the next-hop destination to your current parent's ID, and broadcast the packet. Only that parent processes it.
- Define TTL as the number of transmissions still permitted. A sender requires TTL greater than zero and decrements it before each transmission. A relay receiving TTL zero must discard the packet; the root may accept it because no further transmission is needed.
- Maintain a bounded duplicate cache and suppress repeat forwarding/delivery of the same packet. Define its expiry so that entries outlive the maximum expected packet lifetime.
- If no route exists, count and log the discarded sample or received packet. For this assignment, do not buffer it indefinitely or add application-layer retries.
- At the root, log each unique delivered packet and its origin, sequence, sensor code, and decoded measurement values with units. The root does not forward data.

An ESP-NOW send result is not proof of end-to-end delivery. Only a matching entry at the root counts as successful delivery.

A. Mandatory AES-CTR encryption and integrity checks

All devices must use application-layer AES in counter mode (AES-CTR) with a pre-shared key (PSK), together with CRC-32 or unkeyed SHA-256 for data-integrity checks. Apply encryption and integrity checks to advertisements and data packets in both routing modes. All transmissions remain ESP-NOW broadcasts; implement encryption in the application rather than using individual peer encryption. No authentication mechanism or authentication tag is required or used.

Provision the same randomly generated AES key on participating devices before a run. Agree on one supported AES key size and one integrity method for the entire network. Keep the PSK in a separate local configuration file; never transmit it in packets or print it in logs. Submit only a placeholder configuration and provisioning instructions. Verify AES-CTR and the selected CRC/hash implementation on the actual firmware before implementation.

Use the following outer packet format:

```
version | key_id | sender_id | run_id
| tx_counter | ciphertext
```

Define fixed field widths and byte order for the cleartext outer header. The `sender_id` is the immediate transmitter, and `key_id` selects the locally provisioned PSK. Construct a 16-byte AES-CTR initial counter block from the run identifier, sender ID, transmission counter, and a separate block-counter field. Document the layout and counter increment convention. Reserve enough block-counter values for each packet so that counter blocks never overlap between packets encrypted with the same key.

Before encryption, compute the integrity value over the exact serialized outer header followed by the complete plaintext inner packet. Use either a four-byte CRC-32 (document its variant, parameters, and byte order) or the full 32-byte unkeyed SHA-256 digest. Append this value to the inner packet, then encrypt the combined bytes using AES-CTR. The resulting ciphertext contains both the inner packet, including its sensor-code header, and its integrity value. CTR requires no padding. Report the complete transmitted lengths, including the outer header and integrity value, and ensure they fit the supported ESP-NOW payload limit.

Use a fresh network-wide run identifier for every run. Never reuse AES counter blocks with the same PSK, including after reboots or experiment resets. Persist the required state or provision a fresh PSK before resetting counters; stop before counter wraparound. The per-sender transmission counter covers advertisements, originated data, and relayed data and is separate from the inner sample sequence number. Increment it for every new encryption operation.

On reception, check the outer format, supported version, key ID, current run, and length before decryption. Decrypt, separate the inner packet from the CRC/digest, recompute the integrity value over the received outer header and decrypted inner packet, and compare it with the decrypted value. Discard and count mismatches. Only after a successful check and inner-packet validation may a packet update routing or duplicate state, be forwarded, or enter the root's delivered-data log. Apply the existing DV destination filter at this stage. For advertisements, require the inner source ID to match the outer sender ID as a consistency check.

A relay decrypts and checks the incoming packet, preserves the origin, sample sequence, sensor code, and plaintext sensor payload, and updates the destination and TTL. It then constructs its own outer header, recomputes the CRC/digest, and encrypts with fresh counter blocks before broadcasting. Flooding relays follow the same procedure. Keep the existing bounded sample duplicate cache based on origin and sequence; CRC/SHA-256 checks do not prevent replay.

CRC-32 and unkeyed SHA-256 provide corruption checks, not sender authentication or a guarantee against deliberate packet modification. AES-CTR alone is malleable. Passing these checks must not be described as proof of a packet's origin or authenticity; discuss this limitation in the report.

Demonstration: Deliver encrypted multi-hop sensor data and advertisements successfully. Inject a single-bit error into a captured ciphertext and show that its integrity check fails and that the packet is discarded without changing routing state or reaching the delivered-data log. Confirm that captured broadcasts contain encrypted inner packets rather than readable sensor values. Document the selected CRC/hash method and resulting packet overhead.

VI. Task 4: Threaded operation

Use `_thread`; an implementation using only one sequential loop does not satisfy this requirement. Use at least the following two concurrent execution contexts:

1) **Main/receive thread:** Receive and parse packets, update neighbor information, expire routes, and process or enqueue received data for forwarding.
2) **Worker thread:** Schedule advertisements and locally generated data independently, perform sensor acquisition, and service any outgoing forwarding queue.

You may use additional threads if supported by your board and firmware. Only one thread should consume received ESP-NOW packets. Protect shared neighbor/routing state, queues, duplicate caches, encryption transmission counters and other counters with `_thread.allocate_lock()`. Serialize access to shared broadcast transmit operations, or give one thread ownership of them.

Keep critical sections short. Do not hold a state lock while sleeping, waiting for sensor conversion, or performing a blocking send. Take a consistent snapshot of parent ID and hop count before preparing a packet. If you use a forwarding queue, bound its size and count queue-overflow drops. Add short waits where needed to avoid busy loops.

Demonstration: Show that a device forwards received packets while its own measurements and advertisements continue. Identify one possible race condition and explain how your locking or ownership design prevents it.

VII. Task 5: Flooding baseline

Implement a second mode using the same boards, enabled sensors, payload formats, source IDs, sequence numbers, initial TTL, and data-generation intervals. Both modes use ESP-NOW broadcasts: DV restricts processing to the parent named in the destination field, whereas flooding allows every eligible receiver to relay a new packet. In flooding mode, no parent is selected and no routing advertisements are required.

Broadcast each generated data packet with destination 255. On the first reception of a new packet, a non-root device rebroadcasts it once after a short random delay, provided TTL permits. Preserve origin, sequence, sensor code, and payload bytes; apply the same TTL decrement rule as DV. Mark locally generated packets as seen before broadcasting, and mark received packets as seen before scheduling their relay. Subsequent copies are discarded. The root records each unique packet once and does not rebroadcast it.

Use a bounded duplicate cache with the same lifetime policy as DV. This is duplicate-suppressed flooding, not unrestricted rebroadcasting. Use one broadcast transmission per forwarding event, not a separate unicast to every neighbor.

VIII. Task 6: Experimental comparison

Use a topology with at least one two-hop path and, where possible, two alternative parents for one device. Draw the connectivity graph and report device positions and observed link RSS values. If all boards can reach the root directly, use physical separation to create the topology. Keep the same device positions and radio settings in both modes so that the comparison uses the same connectivity.

Run both protocols in the following scenarios:

1) **Stable topology:** Measure normal delivery and overhead after DV has formed routes.
2) **Topology change:** During the measured run, switch off a relay. Keep the same surviving source set and failure timing in both modes. Observe whether an alternative path exists and whether delivery recovers.

For each protocol and scenario, collect at least three independent runs of at least five minutes. Use the same sensor assignment, per-sensor generation intervals, security scheme, configuration, and offered load in both modes, reset experiment state between runs while preserving AES counter-block uniqueness as required above, and record the warm-up period separately. In DV, allow route formation before the measured interval; then include all advertisements sent during that interval in the overhead counts. Record generated samples even if they are dropped because no route exists. Count one packet as one generated sample regardless of the number of payload fields. Report acquisition failures separately; they are not generated packets and do not enter the PDR denominator. At the end, stop generating new samples and allow a fixed, identical drain interval for pending packets.

Collect source and root logs and per-device counters. Report:

- **Packet delivery ratio (PDR):** Unique packets received by the root divided by samples generated by all non-root sources. Match source/root records by origin and sequence.
- **Per-source PDR:** Show whether some devices are consistently disadvantaged.
- **Transmission overhead:** Total application-level send attempts across all devices divided by unique packets delivered. Include originated data, relayed data, and DV advertisements. Also report data and advertisement counts separately. If nothing is delivered, report the ratio as undefined.
- **Startup convergence time (DV):** Time from a documented common start until every connected device has a valid route that remains unchanged for three advertisement intervals. Measure this before the stable run and report it separately.
- **Failure recovery:** Report the time until an affected surviving source has a valid alternative DV route and the first subsequent packet reaches the root. For both modes, report the gap in root receptions from that source across the failure. If recovery does not occur, state this explicitly.

These are application-level transmission counts; they are not direct measurements of radio airtime or energy. Do not compute one-way latency by subtracting timestamps from unsynchronized ESP32 clocks. If you additionally measure latency, explain your synchronization or common-clock measurement method.

Include at least two plots: per-source PDR for DV versus flooding, and transmission overhead for the two modes in each scenario. Show the mean and variability across runs,

and provide the underlying results table. Explain how RSS filtering, hop count, flooding duplicates, and advertisement overhead affect the results. Do not assume DV will outperform flooding on every metric.

IX. Deliverables

Submit the following through Moodle by the assignment deadline announced there:

1) Complete MicroPython source files for the root and non-root devices, including both routing modes and PSK-based AES-CTR encryption and CRC-32 or SHA-256 integrity checks, with a placeholder key configuration and provisioning instructions. A common configurable program is acceptable.
2) A short report describing the general functionality of the project and how you integrated your sensor to the system, and the encryption format, key provisioning, CTR counter management, CRC/hash checks, corruption tests, and limitations.
3) Raw logs and the script or spreadsheet used to compute metrics and generate plots.

Be ready to demonstrate discovery, parent selection, multi-hop data delivery, concurrent operation, all five sensor payload types decoded at the root, rejection of unknown sensor codes and malformed payloads, and flooding in the class. You will need to explain your own implementation and each team member's contribution.

X. References

- MicroPython ESP-NOW documentation: https://docs.micropython.org/en/v1.29.0/library/espnow.html.
- Espressif ESP-NOW security documentation: https://docs.espressif.com/projects/esp-idf/en/v5.1/esp32/api-reference/network/esp_now.html.
- MicroPython threading documentation: https://docs.micropython.org/en/latest/library/_thread.html.